

빅데이터 분석 시험 대비 이론 정리

1. TF-IDF, Power Law (멱법칙)

TF-IDF (Term Frequency-Inverse Document Frequency)

- 정의: 여러 문서로 이루어진 문서 집합에서 특정 단어가 특정 문서 내에서 얼마나 중요한지를 나타내는 통계적 수치. 문서의 핵심어를 추출하거나, 검색 결과의 순위를 결정하거나, 문서들 사이의 유사도를 구하는 등의 용도로 사용된다.
- 구성 요소:
 - TF (Term Frequency, 단어 빈도):** 특정 문서(d)에서 특정 단어(t)가 등장한 횟수. $TF(t, d) = (\text{문서 } d \text{에서 단어 } t \text{의 등장 횟수}) / (\text{문서 } d \text{의 전체 단어 수})$ 와 같이 정규화하기도 한다.
 - IDF (Inverse Document Frequency, 역문서 빈도):** 특정 단어가 전체 문서 집합에서 얼마나 공통적으로 나타나는지를 나타내는 값의 역수. 드물게 등장하는 단어일수록 IDF 값은 높아진다. $IDF(t, D) = \log((\text{전체 문서 수}) / (\text{단어 } t \text{를 포함한 문서 수} + 1))$. (분모에 1을 더하는 것은 특정 단어가 모든 문서에 등장하여 분모가 0이 되는 것을 방지하기 위함이다).
- 계산: $TF-IDF(t, d, D) = TF(t, d) * IDF(t, D)$
- 의미:
 - 한 문서에서 자주 등장하는 단어(TF가 높음)이면서,
 - 전체 문서 집합에서는 드물게 나타나는 단어(IDF가 높음)일수록
 - TF-IDF 값이 높아지며, 해당 문서의 주제를 잘 나타내는 중요한 단어라고 판단할 수 있다. 예를 들어 '경제' 관련 문서에서는 '금리', '주가' 같은 단어가 TF-IDF가 높게 나오고, 모든 문서에 자주 나오는 '그리고', '하지만' 같은 불용어(stopword)는 IDF가 낮아져 중요도가 낮게 평가된다.

Power Law (멱법칙)

- 정의: 어떤 사건의 발생 빈도나 규모가 그 빈도의 거듭제곱에 반비례하는 관계. 즉, 상위 몇 개의 소수 요소가 전체의 대부분을 차지하는 현상. $f(x) = ax^k$ 형태의 함수로 나타낼 수 있다. (k 는 보통 음수)
- 빅데이터에서의 예시 (**Zipf's Law, 지프의 법칙**):
 - 단어 빈도: 어떤 언어의 말뭉치(corpus)에서 단어의 등장 빈도를 순서대로 나열했을 때, 1위 단어의 빈도는 2위 단어의 약 2배, 3위 단어의 약 3배가 된다는 법칙. 이는 멱법칙의 대표적인 예시이다. ($k \approx -1$)
 - 소수의 단어('the', 'a', 'is' 등)가 텍스트의 대부분을 차지하고, 대다수의 단어는 매우 드물게 사용된다.
- 기타 예시: 웹사이트 방문 횟수(소수의 인기 사이트가 대부분의 트래픽을 차지), 소셜 네트워크의 친구 수(소수의 '인싸'가 다수의 연결을 가짐), 도시의 인구수 등.
- 의미: 빅데이터 분석에서 멱법칙을 따르는 분포를 발견하면, 자원의 불균등한 분포를 이해하고 상위 소수에 집중하는 전략(예: 추천 시스템에서 인기 상품 우선 노출)을 세우거나, 반대로 롱테일(Long Tail)에 해당하는 다수의 비주류 요소에서 새로운 가치를 찾는 전략을 고려할 수 있다.

❓ Q. TF-IDF 계산

문서 A의 내용은 "빅데이터 분석은 빅데이터를 분석하는 것"이고, 총 10개의 문서로 이루어진 문서 집합에서 '분석'이라는 단어는 3개의 문서에 등장한다. 문서 A에서 '분석'의 TF-IDF 점수를 계산하시오. (단, TF는 단순 빈도수, IDF는 $\log(\text{전체 문서 수} / \text{해당 단어 등장 문서 수})$ 로 계산)

② 상세 풀이 >

1. TF (단어 빈도) 계산:

'분석'이라는 단어는 문서 A ("빅데이터 분석은 빅데이터를 분석하는 것")에서 **2번** 등장합니다.
따라서, $TF = 2$ 입니다.

2. IDF (역문서 빈도) 계산:

전체 문서 수는 10개이고, '분석'이 포함된 문서는 3개입니다.

$IDF = \log(10 / 3)$ 입니다.

3. TF-IDF 점수 계산:

$TF-IDF = TF * IDF = 2 * \log(10 / 3)$ 입니다.

2. 분산 시스템의 컴포넌트, 클러스터 컴퓨팅의 한계와 해결 방안

분산 시스템의 주요 컴포넌트

분산 시스템은 여러 컴퓨터(노드)를 네트워크로 연결하여 마치 하나의 컴퓨터처럼 동작하게 만드는 시스템이다. 주요 구성 요소는 다음과 같다.

1. **노드 (Node)**: 시스템을 구성하는 개별 컴퓨터. 연산(CPU), 저장(Disk), 메모리(RAM) 자원을 제공한다.
2. **네트워크 (Network)**: 노드 간의 통신과 데이터 전송을 담당하는 연결망. 속도와 대역폭이 시스템 전체 성능에 큰 영향을 미친다.
3. **분산 파일 시스템 (Distributed File System, e.g., HDFS)**: 대용량 파일을 여러 노드에 분산하여 저장하고 관리한다. 데이터의 복제본(replica)을 유지하여 특정 노드에 장애가 발생해도 데이터 유실을 방지한다(Fault Tolerance).
4. **자원 관리자 (Resource Manager, e.g., YARN)**: 클러스터의 모든 계산 자원을 관리하고, 여러 애플리케이션의 요청에 따라 자원을 할당하고 스케줄링한다.
5. **작업 관리 및 처리 프레임워크 (Processing Framework, e.g., MapReduce, Spark)**: 분산된 데이터를 병렬로 처리하는 방법을 정의하고 실행하는 소프트웨어 계층.

클러스터 컴퓨팅의 한계와 해결 방안

1. 단일 장애점 (Single Point of Failure, SPOF)

- **한계**: 마스터 노드(e.g., HDFS의 NameNode, YARN의 ResourceManager)와 같이 시스템의 핵심적인 역할을 하는 컴포넌트가 고장 나면 전체 시스템이 마비될 수 있다.
- **해결 방안**:
 - **고가용성 (High Availability, HA)**: 마스터 노드를 이중화하여 하나가 실패하면 다른 하나(Standby)가 즉시 역할을 이어받도록 구성한다. (e.g., Active-Standby NameNode 구성)
 - **상태 저장소 공유**: Active/Standby 노드는 ZooKeeper와 같은 코디네이션 서비스를 통해 상태 정보를 공유하고 장애 발생 시 전환(Failover)을 관리한다.

2. 네트워크 병목 (Network Bottleneck)

- **한계**: 셔플(Shuffle)과 같이 노드 간 대규모 데이터 교환이 발생할 때 네트워크 대역폭이 처리량을 따라가지 못해 성능이 저하된다.
- **해결 방안**:
 - **데이터 지역성 (Data Locality) 최적화**: 데이터가 저장된 노드에서 최대한 계산을 수행하여 네트워크를 통한 데이터 이동을 최소화한다. (Spark의 NODE_LOCAL, RACK_LOCAL 스케줄링)
 - **데이터 압축 (Data Compression)**: 전송 전 데이터를 압축하여 네트워크 트래픽 양을 줄인다.
 - **고성능 네트워크 인프라**: 10Gbps 이상의 고대역폭 네트워크 장비(스위치, 랜카드)를 사용한다.

3. 확장성 한계 (Scalability Limit)

- **한계**: 수천, 수만 개의 노드로 확장될수록 마스터 노드가 관리해야 할 메타데이터와 상태 정보가 기하급수적으로 증가하여 마스터 노드 자체가 병목이 된다.
- **해결 방안**:
 - **연합 (Federation)**: 여러 개의 작은 클러스터를 논리적으로 묶어 하나의 큰 클러스터처럼 사용한다. 각 작은 클러스터는 자체 마스터를 가지며, 이를 통해 관리 부하를 분산시킨다. (e.g., HDFS Federation)

4. 데이터 처리의 비효율성 (Inefficient Processing)

- **한계**: MapReduce와 같이 매 연산 단계마다 결과를 디스크에 쓰고 읽는 방식은 I/O 오버헤드가 커서 비효율적이다.
- **해결 방안**:
 - **인메모리 컴퓨팅 (In-Memory Computing)**: Spark처럼 중간 연산 결과를 디스크가 아닌 메모리에 저장(cache, persist)하여 I/O를 최소화하고 반복적인 작업의 속도를 크게 향상시킨다.

HDFS 클러스터의 NameNode에 장애가 발생하여 전체 시스템이 멈췄다. 이 현상을 무엇이라고 부르며, 이를 해결하기 위한 대표적인 아키텍처는 무엇인가?

③ 상세 풀이 >

이 현상은 단일 장애점 (**Single Point of Failure, SPOF**)이라고 부릅니다.

이를 해결하기 위한 대표적인 아키텍처는 고가용성 (**High Availability, HA**) 구성입니다. 구체적으로 NameNode를 이중화하여 Active NameNode와 Standby NameNode를 두는 방식입니다. Active NameNode에 장애가 발생하면, Standby NameNode가 즉시 역할을 넘겨받아(Failover) 서비스 중단을 방지합니다.

3. MapReduce를 통한 WordCount 구현 방식의 차이점

MapReduce에서 WordCount를 구현하는 방식은 Combiner의 사용 여부에 따라 크게 나뉜다.

1. Combiner를 사용하지 않는 기본 방식

- 과정:
 - Input Split**: 입력 데이터(텍스트 파일)가 여러 조각(Split)으로 나뉘어 각 Mapper에게 할당된다.
 - Map** 단계: 각 Mapper는 할당된 텍스트를 읽어 한 줄씩 처리한다. 각 줄을 단어로 분리하고, 모든 단어에 대해 (word, 1) 형태의 Key-Value 쌍을 출력한다.
 - "Hello World Bye World" → (Hello, 1), (World, 1), (Bye, 1), (World, 1)
 - Shuffle and Sort (Grouping)**: Map 단계의 출력 결과가 네트워크를 통해 Reducer로 전송된다. 이 과정에서 동일한 Key(word)를 가진 데이터들이 그룹화되어 같은 Reducer에게 전달된다.
 - Reducer 1 ← (Hello, [1]), (Bye, [1])
 - Reducer 2 ← (World, [1, 1])
 - Reduce** 단계: 각 Reducer는 할당된 Key와 그에 해당하는 값들의 리스트([1, 1, ...])를 입력받아, 값들의 합을 계산하여 최종적인 단어 빈도를 출력한다.
 - Reducer 2 → (World, 2)
- 단점: Map 단계에서 생성된 모든 (word, 1) 쌍이 네트워크를 통해 Reducer로 전송된다. "World" 가 100만 번 나오면 100만 개의 (World, 1) 이 네트워크를 타야 하므로 **네트워크 병목 현상**이 심각하게 발생한다.

2. Combiner를 사용하는 방식

- Combiner란?**: 각 **Mapper** 노드 내에서 Reducer가 수행하는 작업을 미리 한 번 수행하여 네트워크로 전송되는 데이터 양을 줄이는 '미니 Reducer' 역할.
- 과정:
 - Input Split, Map** 단계: 기본 방식과 동일하게 Mapper가 (word, 1) 을 출력한다.
 - Combiner** 단계 (**Mapper** 노드 내에서 수행): Mapper의 출력이 네트워크로 전송되기 전, 같은 **Mapper** 내에서 생성된 동일한 Key를 가진 값들을 미리 합산한다.
 - 한 Mapper가 "Hello World Bye World" 를 처리했다면,
 - Map 출력: (Hello, 1), (World, 1), (Bye, 1), (World, 1)
 - Combiner 출력: (Hello, 1), (Bye, 1), (World, 2)
 - Shuffle and Sort**: Combiner를 거쳐 데이터 양이 줄어든 결과가 Reducer로 전송된다. 위 예시에서 (World, 1) 두 개 대신 (World, 2) 하나만 전송된다.
 - Reduce** 단계: Reducer는 여러 Mapper로부터 받은 Combiner의 출력 결과를 최종적으로 합산한다.
- 장점: 네트워크 트래픽을 크게 줄여 전체적인 작업 성능을 향상시킨다.
- 제약사항: Combiner의 입력/출력 Key-Value 타입은 Reducer의 입력/출력 타입과 동일해야 한다. 또한, 연산이 교환 법칙과 결합 법칙을 만족해야 한다(e.g., 합산, 최댓값). (평균 계산 등에는 바로 적용 불가)

결론적 차이점

구분	Combiner 미사용	Combiner 사용
네트워크 트래픽	매우 많음 (Map의 모든 출력이 전송)	매우 적음 (Mapper 내에서 집계 후 전송)
성능	상대적으로 느림 (네트워크 병목)	상대적으로 빠름
핵심 아이디어	중앙 집중형 집계 (Reducer에서만 합산)	분산형 사전 집계 (Mapper 로컬에서 1차 합산)

🔗 Q. Combiner의 역할

한 Mapper가 처리하는 데이터에 "spark"라는 단어가 100번 등장했다. Combiner를 사용하지 않았을 때와 사용했을 때, "spark"와 관련된 데이터가 네트워크를 통해 Reducer로 전송되는 과정의 차이점을 설명하시오.

🔗 상세 풀이 >

- **Combiner 미사용 시:** Mapper는 ("spark", 1)이라는 Key-Value 쌍을 100개 생성하고, 이 100개의 쌍이 모두 네트워크를 통해 Reducer로 전송됩니다.
- **Combiner 사용 시:** Mapper는 ("spark", 1) 100개를 생성한 후, Mapper 노드 내의 Combiner가 이를 미리 집계하여 ("spark", 100)이라는 하나의 Key-Value 쌍으로 만듭니다. 따라서 네트워크를 통해서는 이 1개의 쌍만 Reducer로 전송됩니다.

결론적으로, **Combiner** 사용 시 네트워크 트래픽이 **100개에서 1개로** 크게 감소합니다.

4. Natural Join에 대한 Relational Algebra, 필요한 MapReduce Job 수

Natural Join (자연 조인)과 관계 대수 (Relational Algebra)

- 정의: 두 릴레이션(테이블)에서 공통된 속성(attribute, 컬럼) 이름을 기준으로, 해당 속성의 값이 동일한 튜플(행)들을 연결하는 연산.
- 관계 대수 표현: $R \bowtie S$
- 동작 방식:
 1. 두 릴레이션 R과 S의 공통 속성을 찾는다.
 2. R과 S의 튜플들을 조합하되, 공통 속성의 값이 같은 것들만 남긴다.
 3. 결과 릴레이션에서는 공통 속성이 한 번만 나타난다.
- 예시:
 - 릴레이션 Student(ID, Name)
 - 릴레이션 Takes(ID, Course)
 - $Student \bowtie Takes$
 - 공통 속성은 ID이다.
 - 두 릴레이션에서 ID 값이 같은 튜플들을 결합하여 (ID, Name, Course) 형태의 새로운 릴레이션을 만든다.

Natural Join을 위한 MapReduce Job의 수

Natural Join은 하나의 MapReduce Job으로 구현할 수 있다. (Reduce-side Join 방식)

- 필요한 Job 수: 1개
- 구현 방식 (Reduce-side Join):
 1. Map 단계:
 - 두 릴레이션(R, S)의 모든 튜플을 읽는다.
 - 조인의 기준이 되는 공통 속성의 값을 Key로 설정한다.

- Value에는 릴레이선의 출처를 구분할 수 있는 태그(tag)와 튜플의 나머지 데이터를 담는다.

- 예시 Student $\bowtie$ Takes :

- Student 릴레이선의 $(1, 'Alice') \rightarrow (1, ('Student', 'Alice'))$
- Takes 릴레이선의 $(1, 'CS101') \rightarrow (1, ('Takes', 'CS101'))$

2. Shuffle and Sort:

- Map의 출력들이 Key(ID)를 기준으로 그룹화되어 동일한 Key를 가진 데이터가 같은 Reducer로 보내진다.
- Reducer $\leftarrow (1, [('Student', 'Alice'), ('Takes', 'CS101'), ('Takes', 'CS102'), \dots])$

3. Reduce 단계:

- Reducer는 동일한 Key에 대해 그룹화된 값 리스트를 입력받는다.
- 리스트 내의 값들을 순회하면서, 태그를 기준으로 서로 다른 릴레이선에서 온 튜플들을 찾아 조합(Cartesian Product)한다.
- 예시: Key 1 에 대해 Reducer는 ('Student', 'Alice') 를 발견하고, 같은 Key에 속한 ('Takes', ...) 값들을 찾는다.
 - ('Student', 'Alice') 와 ('Takes', 'CS101') 을 조합하여 최종 결과 $(1, 'Alice', 'CS101')$ 을 출력한다.
 - ('Student', 'Alice') 와 ('Takes', 'CS102') 를 조합하여 $(1, 'Alice', 'CS102')$ 를 출력한다.
- 만약 한 릴레이선에서만 온 데이터만 있다면 (짜이 없다면) 조인은 이루어지지 않고 버려진다.

- 여러 개의 릴레이선을 조인할 경우?

- $R \bowtie S \bowtie T$ 와 같이 세 개 이상의 릴레이선을 조인할 때는 여러 개의 MapReduce Job이 필요할 수 있다.
- 가장 간단한 방법은 두 릴레이선을 먼저 조인하고(Job 1), 그 결과와 나머지 릴레이선을 조인하는(Job 2) 식으로 연쇄적으로 수행하는 것이다. 이 경우 Job의 개수는 (릴레이선 수 - 1) 개가 된다.

Q. MapReduce Join

Employee(ID, Name) 릴레이선과 Department(ID, DeptName) 릴레이선을 MapReduce를 이용해 Natural Join 하려고 한다. Map 단계에서 Employee 의 $(10, 'John')$ 튜플은 어떤 Key-Value 쌍으로 변환되어야 하는가?

상세 풀이 >

조인 키(Join Key)인 ID 를 Map의 출력 Key로 사용하고, Value에는 릴레이선 출처를 알리는 태그와 나머지 데이터를 담습니다.

따라서 Key는 10 이 되고, Value는 ('Employee', 'John') 과 같은 형태의 쌍으로 변환됩니다.

최종적인 Map 출력 형태는 $(10, ('Employee', 'John'))$ 입니다.

5. 대치/보간법, PCA, 지니불순도/엔트로피, IV/WoE, Monotonicity

- 대치/보간법 (Imputation/Interpolation): 데이터의 결측치(missing value)를 적절한 값으로 채우는 기법.
 - 대치: 평균, 중앙값, 최빈값 등 통계량을 사용하거나, 다른 변수와의 관계를 이용한 예측 모델(e.g., 회귀)로 결측치를 채운다.
 - 보간: 시계열 데이터에서 주로 사용되며, 결측된 지점의 앞뒤 데이터 값을 이용하여 중간값을 추정한다. (e.g., 선형 보간)
- PCA 결과 분석 (Principal Component Analysis, 주성분 분석):
 - 정의: 고차원 데이터의 분산을 최대한 보존하는 새로운 저차원 좌표축(주성분)을 찾아 데이터를 변환하는 차원 축소 기법.
 - 결과 분석:
 - 고유값(Eigenvalue): 각 주성분이 설명하는 데이터 분산의 크기. 고유값이 클수록 중요한 주성분이다.
 - 스크리 그림(Scree Plot): 고유값을 크기 순으로 시각화한 그래프. 그래프가 완만해지기 시작하는 지점(Elbow)을 기준으로 사용할 주성분의 개수를 결정한다.
 - 누적 설명 분산(Cumulative Explained Variance): 상위 k개의 주성분이 전체 데이터 분산의 몇 %를 설명하는지 나타내는 지표. 보통 80~95% 이상이 되는 지점까지의 주성분을 선택한다.

- 지니 불순도(**Gini Impurity**) / 엔트로피(**Entropy**): 의사결정나무(Decision Tree)에서 분기(split)의 기준을 정하기 위해 사용되는 불순도(Impurity) 측정 지표. 값이 낮을수록 순도가 높다(잘 분류됨).
 - 지니 불순도: $Gini = 1 - \sum (p_i)^2$. (p_i : i번째 클래스에 속할 확률). 계산이 빨라 주로 사용된다.
 - 엔트로피: $Entropy = -\sum (p_i) * \log_2(p_i)$. 정보 이론의 개념으로, 불확실성의 정도를 나타낸다.
 - 정보 이득(**Information Gain**): 분기 전의 불순도 - 분기 후의 가중 평균 불순도. 정보 이득이 가장 큰 변수와 분기점을 선택한다.
- **IV (Information Value) / WoE (Weight of Evidence)**: 신용 평가 모델링 등에서 변수의 예측력을 평가하고 범주형 변수를 수치화하기 위해 사용되는 기법.
 - **WoE**: 특정 범주가 전체의 Good/Bad 비율에 비해 얼마나 Good/Bad를 잘 예측하는지를 나타내는 값. $WoE = \ln(\text{전체 Good 중 해당 범주의 비율} / \text{전체 Bad 중 해당 범주의 비율})$
 - **IV**: 각 범주의 WoE 값에 해당 범주의 Good/Bad 비율 차이를 가중치로 곱하여 합산한 값. 변수 전체의 예측력을 나타낸다. IV가 높을수록 예측력이 강한 변수이다.
- 단조성 (**Monotonicity**): 한 변수가 증가할 때 다른 변수도 꾸준히 증가하거나(단조 증가) 꾸준히 감소하는(단조 감소) 경향성.
 - 중요성: 신용 평가 모델 등에서 매우 중요하다. 예를 들어 '소득'이 증가할수록 '연체율'은 꾸준히 감소해야 상식에 부합하고 모델이 안정적이라고 본다.
 - 검증: 변수를 구간화(binning)하고 각 구간별 WoE 값이나 불량률이 단조적인 경향을 보이는지 확인한다. 스피어만 순위 상관계수로 측정하기도 한다.

🔗 Q. PCA 결과 분석

PCA를 수행하여 3개의 주성분(PC1, PC2, PC3)을 얻었고, 각각의 고유값(Eigenvalue)이 5.2, 2.7, 0.1 이었다. 이 때, 상위 2개의 주성분(PC1, PC2)만으로 설명되는 데이터의 총 분산 비율(누적 설명 분산)은 얼마인가?

🔗 상세 풀이 >

1. 전체 분산 (총 에너지) 계산:

모든 고유값의 합 = $5.2 + 2.7 + 0.1 = 8.0$

2. 상위 2개 주성분이 설명하는 분산 계산:

$5.2 + 2.7 = 7.9$

3. 누적 설명 분산 비율 계산:

비율 = (상위 2개 주성분 분산) / (전체 분산) = $7.9 / 8.0 = 0.9875$

따라서 누적 설명 분산은 **98.75%** 입니다.

6. Map vs FlatMap, Persistence, Cache, Broadcast, Accumulator, Lazy Evaluation

- **Map vs. FlatMap (Spark RDD/DataFrame 연산)**:
 - **Map**: RDD의 각 요소에 함수를 적용하여 **1:1**로 새로운 요소를 만들어내는 변환(transformation). 입력 요소 하나당 출력 요소 하나가 생성된다. $RDD[T] \rightarrow RDD[U]$
 - `[ "hello world", "bye world" ] → map(_.split(" ")) → [ ["hello", "world"], ["bye", "world"] ] (Array[Array[String]])`
 - **FlatMap**: RDD의 각 요소에 함수를 적용하여 **1:N** (0개 이상)의 새로운 요소들을 만들어내고, 그 결과들을 하나의 **RDD**로 펼쳐서(flatten) 반환하는 변환. $RDD[T] \rightarrow RDD[U]$
 - `[ "hello world", "bye world" ] → flatMap(_.split(" ")) → [ "hello", "world", "bye", "world" ] (Array[String])`
- **Persistence (persist()) / cache()**: Spark에서 RDD나 DataFrame을 메모리나 디스크에 저장하여 재사용 시 성능을 높이는 최적화 기법.
 - **cache()**: `persist(StorageLevel.MEMORY_ONLY)` 와 동일. RDD를 메모리에만 저장한다.
 - **persist()**: 다양한 저장 레벨(메모리, 디스크, 둘 다 사용 등)을 직접 지정할 수 있다.

- 동작: 최초의 액션(action) 연산이 호출될 때 해당 RDD가 계산되면서 그 결과가 지정된 저장소에 저장된다. 이후 동일한 RDD를 사용하는 작업은 처음부터 다시 계산하지 않고 저장된 데이터를 읽어오므로 속도가 매우 빠르다.
- **Broadcast Variable (방송 변수):** 드라이버(Driver) 프로그램에 있는 비교적 작은 크기의 데이터(e.g., lookup table, feature list)를 모든 Executor 노드에 효율적으로 전송하기 위한 읽기 전용 공유 변수.
 - 장점: 일반 변수를 사용하면 Task마다 해당 변수가 네트워크를 통해 복제되어 전송되지만, 방송 변수는 Executor 당 한 번만 전송되므로 네트워크 부하를 크게 줄일 수 있다.
- **Accumulator (누산기):** 여러 Task에서 발생한 값을 드라이버 프로그램에서 합산(aggregate)하기 위한 공유 변수. 덧셈이나 카운팅 등에 사용된다.
 - 특징: Executor는 누산기에 값을 더할 수만 있고 읽을 수는 없다(Write-only). 오직 드라이버만이 누산기의 최종 값을 읽을 수 있다. 분산 환경에서 안정적으로 집계 연산을 수행할 수 있도록 보장한다. (e.g., 에러 라인 수 카운팅)
- **Lazy Evaluation (지연 실행):** Spark의 핵심 동작 원리.
 - 정의: 변환(Transformation) 연산(map , filter 등)은 호출 즉시 실행되지 않고, 실행 계획(DAG, Directed Acyclic Graph)에 기록만 된다. 실제 계산은 액션(Action) 연산(count , collect , save 등)이 호출되었을 때 비로소 시작된다.
 - 장점:
 - 최적화: 여러 변환 연산을 파이프라이닝(pipelining)하여 한 번에 처리하는 등 전체적인 실행 계획을 최적화할 수 있다.
 - 효율성: 불필요한 중간 데이터 생성을 방지하고 메모리 사용을 줄인다.

🔗 Q. Map vs FlatMap

RDD `rdd = sc.parallelize(["a b", "c d e"])` 가 있을 때, `rdd.map(_.split(" "))` 의 결과와 `rdd.flatMap(_.split(" "))` 의 결과는 각각 무엇인가?

🔗 상세 풀이 >

- **map 결과:** map 은 각 원소를 1:1로 변환하므로, 각 문자열이 단어들의 '배열'로 변환된 새로운 RDD가 생성됩니다.
 - 결과 RDD의 타입: `RDD[Array[String]]`
 - 내부 데이터 형태: `[ ["a", "b"], ["c", "d", "e"] ]`
- **flatMap 결과:** flatMap 은 변환 결과들을 하나의 RDD로 펼쳐주므로, 모든 단어들이 포함된 단일 RDD가 생성됩니다.
 - 결과 RDD의 타입: `RDD[String]`
 - 내부 데이터 형태: `[ "a", "b", "c", "d", "e" ]`

7. MLflow의 4가지 구성요소, Fundamental Modules, Data manipulation

MLflow의 4가지 구성요소 (Components)

MLflow는 머신러닝 모델 개발의 전체 생명주기(lifecycle)를 관리하기 위한 오픈소스 플랫폼이다.

1. MLflow Tracking:

- 역할: 실험(experiment)을 기록하고 관리하는 컴포넌트.
- 기능: 모델 학습 시 사용된 파라미터(parameters), 성능 지표(metrics), 소스 코드 버전, 그리고 생성된 모델 아티팩트(artifacts) 등을 자동으로 기록하고 비교할 수 있게 해준다.

2. MLflow Projects:

- 역할: 머신러닝 코드를 재현 가능하고 재사용 가능한 형태로 패키징하는 규약(convention).
- 기능: MLproject 라는 설정 파일을 통해 코드의 의존성(e.g., conda.yaml), 실행 진입점(entry points)을 정의한다. 이를 통해 다른 환경이나 다른 개발자도 mlflow run 명령어로 동일한 실험을 쉽게 재현할 수 있다.

3. MLflow Models:

- 역할: 다양한 머신러닝 라이브러리(e.g., Scikit-learn, TensorFlow, PyTorch)로 학습된 모델을 패키징하고 배포하기 위한 표준 포맷.

- 기능: MLmodel 파일을 통해 모델을 서빙하는 방법을 정의한다. 이를 통해 로컬 REST API, 클라우드 플랫폼(Azure ML, AWS SageMaker) 등 다양한 환경에 일관된 방식으로 모델을 배포할 수 있다.

4. MLflow Model Registry:

- 역할: 학습된 모델의 전체 생명주기를 중앙에서 관리하는 저장소.
- 기능: 모델의 버전 관리, 스테이지 전환(Staging → Production → Archived), 모델에 대한 설명 및 태깅 등을 지원한다. 이를 통해 CI/CD 파이프라인과 연계하여 모델의 배포 및 관리를 자동화할 수 있다.

(질문 의도에 따라) **Fundamental Modules / Data Manipulation**은 **Spark** 관련 내용일 수 있어 함께 정리

Spark의 Fundamental Modules

- **Spark Core**: Spark의 가장 핵심적인 엔진. RDD(Resilient Distributed Dataset) 개념을 기반으로 분산 데이터 처리, 메모리 관리, 장애 복구, 스케줄링 등의 기본 기능을 제공한다.
- **Spark SQL**: 정형 데이터(structured data)를 다루기 위한 모듈. DataFrame과 Dataset API를 제공하며, 표준 SQL 쿼리를 사용하여 데이터를 처리할 수 있다.
- **Spark Streaming**: 실시간 스트리밍 데이터를 처리하기 위한 모듈. DStream(Discretized Stream)이라는 개념을 통해 작은 배치(micro-batch) 단위로 데이터를 처리한다. (최근에는 구조적 스트리밍(Structured Streaming)이 주로 사용됨)
- **MLlib (Machine Learning Library)**: 스파크 위에서 동작하는 분산 머신러닝 라이브러리. 분류, 회귀, 클러스터링, 추천 등 다양한 알고리즘을 제공한다.
- **GraphX**: 그래프(Graph) 데이터 구조와 그래프 병렬 처리 알고리즘을 제공하는 라이브러리.

Spark의 Data Manipulation (데이터 조작)

- **Transformation (변환)**: RDD나 DataFrame을 입력받아 새로운 RDD/DataFrame을 반환하는 연산. (`map`, `filter`, `join`, `groupBy`)
- **Action (액션)**: 실제 계산을 실행하고 결과를 반환하거나 저장하는 연산. (`count`, `collect`, `save`, `foreach`)
- **DataFrame/Dataset API**: `select`, `where`, `groupBy`, `agg`, `withColumn` 등 SQL과 유사한 고수준 API를 통해 데이터를 조작한다. 이는 Spark SQL의 Catalyst Optimizer를 통해 최적화된 코드로 실행된다.

❓ Q. MLflow 컴포넌트 선택

여러 하이퍼파라미터 조합으로 모델을 학습시키면서 각 실험의 성능 지표(e.g., accuracy)와 사용된 파라미터를 기록하고, 나중에 최적의 실험을 찾기 위해 비교하고 싶다. MLflow의 어떤 컴포넌트를 사용해야 하는가?

❓ 상세 풀이 >

MLflow Tracking 컴포넌트를 사용해야 합니다.

MLflow Tracking은 실험의 파라미터(parameters), 메트릭(metrics), 아티팩트(artifacts) 등을 서버에 기록하고, UI를 통해 각 실험 결과를 시각적으로 확인하고 비교할 수 있는 기능을 제공하기 때문입니다.

8. 다목적 최적화 (Multi-objective Optimization)

- 정의: 서로 상충(trade-off) 관계에 있는 두 개 이상의 목적 함수(objective function)를 동시에 최적화하는 문제를 다루는 분야.
 - 예시: 자동차 설계 시 '성능'은 최대로, '연비'는 최대로(비용은 최소로), '안전성'은 최대로 하는 것. 성능을 높이면 연비가 나빠지는 등 목적 함수 간에 상충 관계가 존재한다.
- 우월성 (Dominance):
 - 여러 해(solution)들 중에서 어느 해가 더 나은지를 비교하는 개념.
 - 해 A가 해 B를 '지배한다' 또는 '우월하다'는 것은, 모든 목적 함수에 대해 A의 성능이 B보다 나쁘지 않으면서(크거나 같으면서), 적어도 하나의 목적 함수에서는 A의 성능이 B보다 명백히 더 좋을 때를 의미한다.
- 파레토 최적 집합 (Pareto Optimal Set):

- 어떠한 다른 해에 의해서도 지배되지 않는 해(non-dominated solution)들의 집합.
- 이 집합에 속한 해들은 서로 우열을 가릴 수 없다. 즉, 하나의 목적 함수 값을 개선하려면 반드시 다른 목적 함수 값이 나빠져야만 하는 상태의 해들이다.
- **파레토 최적 전선 (Pareto Optimal Front):**
 - 파레토 최적 집합에 속한 해들을 목적 함수 공간(objective space)에 시각화했을 때 나타나는 선 또는 면.
 - 의사 결정자는 이 파레토 Front를 보고, 여러 상충 관계를 고려하여 최종적으로 어떤 해를 선택할지 결정하게 된다.
- **가중합 방법 (Weighted Sum Method):**
 - 다목적 최적화 문제를 단일 목적 최적화 문제로 변환하는 가장 간단한 방법.
 - 각 목적 함수에 가중치(weight) w_i 를 곱하여 모두 더한 하나의 함수 $\sum(w_i * f_i(x))$ 를 만들어 이를 최적화한다. 가중치의 합은 보통 1이 되도록 설정한다. ($\sum w_i = 1$)
 - **장점:**
 - 구현이 매우 간단하고 직관적이다.
 - 기존의 단일 목적 최적화 알고리즘을 그대로 활용할 수 있다.
 - **단점:**
 - **적절한 가중치 설정이 어렵다:** 가중치는 각 목적 함수의 상대적 중요도를 나타내는데, 이를 사전에 결정하기가 매우 어렵다.
 - **오목한(non-convex) 파레토 전선 문제:** 파레토 전선이 오목한 형태일 경우, 가중치를 어떻게 조절해도 오목한 부분에 해당하는 해들을 찾을 수 없다.

❓ Q. 우월성(Dominance) 관계 판단

두 개의 목적 함수(f_1, f_2)를 최대화하는 문제에서 두 개의 해(Solution) A와 B가 있다.

- Solution A: ($f_1=10, f_2=20$)
 - Solution B: ($f_1=10, f_2=15$)
- 두 해의 우월성(Dominance) 관계를 설명하시오.

❓ 상세 풀이 >

해 A는 f_1 값(10)에서 해 B의 f_1 값(10)과 동일하고, f_2 값(20)에서는 해 B의 f_2 값(15)보다 우수합니다.

즉, 해 A는 모든 목적 함수에서 B보다 나쁘지 않으면서(f_1), 적어도 하나의 목적 함수에서 B보다 명백히 더 좋습니다(f_2).

따라서 "해 A가 해 B를 지배한다 (Solution A dominates Solution B)"고 말할 수 있습니다.